

Explain what is wrong with the “obvious” approach to polynomial division using a point-value representation: dividing the corresponding y values. Discuss separately the case in which the division comes out exactly and the case in which it doesn’t.

30.1-7

Consider two sets A and B , each having n integers in the range from 0 to $10n$. The **Cartesian sum** of A and B is defined by

$$C = \{x + y : x \in A \text{ and } y \in B\}.$$

The integers in C lie in the range from 0 to $20n$. Show how, in $O(n \lg n)$ time, to find the elements of C and the number of times each element of C is realized as a sum of elements in A and B . (*Hint*: Represent A and B as polynomials of degree at most $10n$.)

30.2 The DFT and FFT

In Section 30.1, we claimed that by computing the DFT and its inverse by using the FFT, it is possible to evaluate and interpolate a degree n polynomial at the complex roots of unity in $\Theta(n \lg n)$ time. This section defines complex roots of unity, studies their properties, defines the DFT, and then shows how the FFT computes the DFT and its inverse in $\Theta(n \lg n)$ time.

Complex roots of unity

A **complex n th root of unity** is a complex number ω such that

$$\omega^n = 1.$$

There are exactly n complex n th roots of unity: $e^{2\pi i k/n}$ for $k = 0, 1, \dots, n - 1$. To interpret this formula, use the definition of the exponential of a complex number:

$$e^{iu} = \cos(u) + i \sin(u).$$

Figure 30.2 shows that the n complex roots of unity are equally spaced around the circle of unit radius centered at the origin of the complex plane. The value

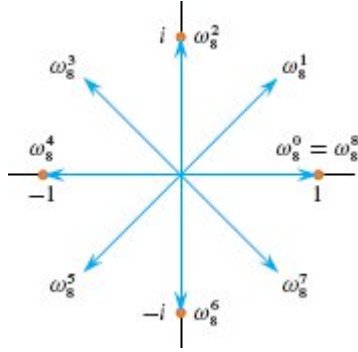


Figure 30.2 The values of $\omega_8^0, \omega_8^1, \dots, \omega_8^7$ in the complex plane, where $\omega_8 = e^{2\pi i/8}$ is the principal 8th root of unity.

$$\omega_n = e^{2\pi i/n} \quad (30.6)$$

is the *principal n th root of unity*.² All other complex n th roots of unity are powers of ω_n .

The n complex n th roots of unity,

$$\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1},$$

form a group under multiplication (see Section 31.3). This group has the same structure as the additive group $(\mathbb{Z}_n, +)$ modulo n , since $\omega_n^n = \omega_n^0 = 1$ implies that $\omega_n^j \omega_n^k = \omega_n^{j+k} = \omega_n^{(j+k) \bmod n}$. Similarly, $\omega_n^{-1} = \omega_n^{n-1}$. The following lemmas furnish some essential properties of the complex n th roots of unity.

Lemma 30.3 (Cancellation lemma)

For any integers $n > 0$, $k \geq 0$, and $d > 0$,

$$\omega_{dn}^{dk} = \omega_n^k. \quad (30.7)$$

Proof The lemma follows directly from equation (30.6), since

$$\begin{aligned}
 \omega_{dn}^{dk} &= (e^{2\pi i/dn})^{dk} \\
 &= (e^{2\pi i/n})^k \\
 &= \omega_n^k.
 \end{aligned}$$

■

Corollary 30.4

For any even integer $n > 0$,

$$\omega_n^{n/2} = \omega_2 = -1.$$

Proof The proof is left as Exercise 30.2-1.

■

Lemma 30.5 (Halving lemma)

If $n > 0$ is even, then the squares of the n complex n th roots of unity are the $n/2$ complex $(n/2)$ th roots of unity.

Proof By the cancellation lemma, $(\omega_n^k)^2 = \omega_{n/2}^k$ for any nonnegative integer k . Squaring all of the complex n th roots of unity produces each $(n/2)$ th root of unity exactly twice, since

$$\begin{aligned}
 (\omega_n^{k+n/2})^2 &= \omega_n^{2k+n} \\
 &= \omega_n^{2k} \omega_n^n \\
 &= \omega_n^{2k} \\
 &= (\omega_n^k)^2.
 \end{aligned}$$

Thus ω_n^k and $\omega_n^{k+n/2}$ have the same square. We could also have used Corollary 30.4 to prove this property, since $\omega_n^{n/2} = -1$ implies $\omega_n^{k+n/2} = \omega_n^k \omega_n^{n/2} = -\omega_n^k$, and thus $(\omega_n^{k+n/2})^2 = (-\omega_n^k)^2 = (\omega_n^k)^2$.

■

As we'll see, the halving lemma is essential to the divide-and-conquer approach for converting between coefficient and point-value representations of polynomials, since it guarantees that the recursive subproblems are only half as large.

Lemma 30.6 (Summation lemma)

For any integer $n \geq 1$ and nonzero integer k not divisible by n ,

$$\sum_{j=0}^{n-1} (\omega_n^k)^j = 0.$$

Proof Equation (A.6) on page 1142 applies to complex values as well as to reals, giving

$$\begin{aligned} \sum_{j=0}^{n-1} (\omega_n^k)^j &= \frac{(\omega_n^k)^n - 1}{\omega_n^k - 1} \\ &= \frac{(\omega_n^n)^k - 1}{\omega_n^k - 1} \\ &= \frac{(1)^k - 1}{\omega_n^k - 1} \\ &= 0. \end{aligned}$$

To see that the denominator is not 0, note that $\omega_n^k = 1$ only when k is divisible by n , which the lemma statement prohibits. ■

The DFT

Recall the goal of evaluating a polynomial

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

of degree-bound n at $\omega_n^0, \omega_n^1, \omega_n^2, \dots, \omega_n^{n-1}$ (that is, at the n complex n th roots of unity).³ The polynomial A is given in coefficient form: $a = (a_0, a_1, \dots, a_{n-1})$. Let us define the results y_k , for $k = 0, 1, \dots, n-1$, by

$$\begin{aligned} y_k &= A(\omega_n^k) \\ &= \sum_{j=0}^{n-1} a_j \omega_n^{kj}. \end{aligned} \tag{30.8}$$

The vector $y = (y_0, y_1, \dots, y_{n-1})$ is the *discrete Fourier transform (DFT)* of the coefficient vector $a = (a_0, a_1, \dots, a_{n-1})$. We also write $y = \text{DFT}_n(a)$.

The FFT

The *fast Fourier transform (FFT)* takes advantage of the special properties of the complex roots of unity to compute $\text{DFT}_n(a)$ in $\Theta(n \lg n)$ time, as opposed to the $\Theta(n^2)$ time of the straightforward method. Assume throughout that n is an exact power of 2. Although strategies for dealing with sizes that are not exact powers of 2 are known, they are beyond the scope of this book.

The FFT method employs a divide-and-conquer strategy, using the even-indexed and odd-indexed coefficients of $A(x)$ separately to define the two new polynomials $A^{\text{even}}(x)$ and $A^{\text{odd}}(x)$ of degree-bound $n/2$:

$$A^{\text{even}}(x) = a_0 + a_2x + a_4x^2 + \cdots + a_{n-2}x^{n/2-1},$$

$$A^{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + \cdots + a_{n-1}x^{n/2-1}.$$

Note that A^{even} contains all the even-indexed coefficients of A (the binary representation of the index ends in 0) and A^{odd} contains all the odd-indexed coefficients (the binary representation of the index ends in 1). It follows that

$$A(x) = A^{\text{even}}(x^2) + xA^{\text{odd}}(x^2), \quad (30.9)$$

so that the problem of evaluating $A(x)$ at $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$ reduces to

1. evaluating the degree-bound $n/2$ polynomials $A^{\text{even}}(x)$ and $A^{\text{odd}}(x)$ at the points

$$(\omega_n^0)^2, (\omega_n^1)^2, \dots, (\omega_n^{n-1})^2, \quad (30.10)$$

and then

2. combining the results according to equation (30.9).

By the halving lemma, the list of values (30.10) consists not of n distinct values but only of the $n/2$ complex $(n/2)$ th roots of unity, with each root occurring exactly twice. Therefore, the FFT recursively evaluates the polynomials A^{even} and A^{odd} of degree-bound $n/2$ at the $n/2$ complex $(n/2)$ th roots of unity. These subproblems have exactly the same form as the original problem, but are half the size, dividing an n -

element DFT_n computation into two $n/2$ -element $\text{DFT}_{n/2}$ computations. This decomposition is the basis for the FFT procedure on the next page, which computes the DFT of an n -element vector $a = (a_0, a_1, \dots, a_{n-1})$, where n is an exact power of 2.

The FFT procedure works as follows. Lines 1–2 represent the base case of the recursion. The DFT of 1 element is the element itself, since in this case

$$\begin{aligned} y_0 &= a_0 \omega_1^0 \\ &= a_0 \cdot 1 \\ &= a_0. \end{aligned}$$

Lines 5–6 define the coefficient vectors for the polynomials A^{even} and A^{odd} . Lines 3, 4, and 12 guarantee that ω is updated properly so that whenever lines 10–11 are executed, $\omega = \omega_n^k$. (Keeping a running value of ω from iteration to iteration saves time over computing ω_n^k from scratch each time through the **for** loop.⁴) Lines 7–8 perform the recursive $\text{DFT}_{n/2}$ computations, setting, for $k = 0, 1, \dots, n/2 - 1$,

```

FFT( $a, n$ )
1 if  $n == 1$ 
2   return  $a$                                 // DFT of 1 element is the element itself
3  $\omega_n = e^{2\pi i/n}$ 
4  $\omega = 1$ 
5  $a^{\text{even}} = (a_0, a_2, \dots, a_{n-2})$ 
6  $a^{\text{odd}} = (a_1, a_3, \dots, a_{n-1})$ 
7  $y^{\text{even}} = \text{FFT}(a^{\text{even}}, n/2)$ 
8  $y^{\text{odd}} = \text{FFT}(a^{\text{odd}}, n/2)$ 
9 for  $k = 0$  to  $n/2 - 1$                       // at this point,  $\omega = \omega_n^k$ 
10    $y_k = y_k^{\text{even}} + \omega y_k^{\text{odd}}$ 
11    $y_{k+(n/2)} = y_k^{\text{even}} - \omega y_k^{\text{odd}}$ 

```

12 $\omega = \omega \omega_n$

13 **return** y

$$y_k^{\text{even}} = A^{\text{even}}(\omega_{n/2}^k),$$

$$y_k^{\text{odd}} = A^{\text{odd}}(\omega_{n/2}^k),$$

or, since $\omega_{n/2}^k = \omega_n^{2k}$ by the cancellation lemma,

$$y_k^{\text{even}} = A^{\text{even}}(\omega_n^{2k}),$$

$$y_k^{\text{odd}} = A^{\text{odd}}(\omega_n^{2k}).$$

Lines 10–11 combine the results of the recursive $\text{DFT}_{n/2}$ calculations. For the first $n/2$ results $y_0, y_1, \dots, y_{n/2-1}$, line 10 yields

$$\begin{aligned} y_k &= y_k^{\text{even}} + \omega_n^k y_k^{\text{odd}} \\ &= A^{\text{even}}(\omega_n^{2k}) + \omega_n^k A^{\text{odd}}(\omega_n^{2k}) \\ &= A(\omega_n^k) \quad (\text{by equation (30.9)}). \end{aligned}$$

For $y_{n/2}, y_{n/2+1}, \dots, y_{n-1}$, letting $k = 0, 1, \dots, n/2 - 1$, line 11 yields

$$\begin{aligned} y_{k+(n/2)} &= y_k^{\text{even}} - \omega_n^k y_k^{\text{odd}} \\ &= y_k^{\text{even}} + \omega_n^{k+(n/2)} y_k^{\text{odd}} \quad (\text{since } \omega_n^{k+(n/2)} = -\omega_n^k) \\ &= A^{\text{even}}(\omega_n^{2k}) + \omega_n^{k+(n/2)} A^{\text{odd}}(\omega_n^{2k}) \\ &= A^{\text{even}}(\omega_n^{2k+n}) + \omega_n^{k+(n/2)} A^{\text{odd}}(\omega_n^{2k+n}) \quad (\text{since } \omega_n^{2k+n} = \omega_n^{2k}) \\ &= A(\omega_n^{k+(n/2)}) \quad (\text{by equation (30.9)}). \end{aligned}$$

Thus the vector y returned by FFT is indeed the DFT of the input vector a .

Lines 10 and 11 multiply each value y_k^{odd} by ω_n^k , for $k = 0, 1, \dots, n/2 - 1$. Line 10 adds this product to y_k^{even} , and line 11 subtracts it. Because each factor ω_n^k appears in both its positive and negative forms, we call the factors ω_n^k *twiddle factors*.

To determine the running time of the procedure FFT, note that exclusive of the recursive calls, each invocation takes $\Theta(n)$ time, where n is the length of the input vector. The recurrence for the running time is therefore

$$\begin{aligned} T(n) &= 2T(n/2) + \Theta(n) \\ &= \Theta(n \lg n), \end{aligned}$$

by case 2 of the master theorem (Theorem 4.1). Thus the FFT can evaluate a polynomial of degree-bound n at the complex n th roots of unity in $\Theta(n \lg n)$ time.

Interpolation at the complex roots of unity

The polynomial multiplication scheme entails converting from coefficient form to point-value form by evaluating the polynomial at the complex roots of unity, pointwise multiplying, and finally converting from point-value form back to coefficient form by interpolating. We've just seen how to evaluate, so now we'll see how to interpolate the complex roots of unity by a polynomial. To interpolate, we'll write the DFT as a matrix equation and then look at the form of the matrix inverse.

From equation (30.4), we can write the DFT as the matrix product $y = V_n a$, where V_n is a Vandermonde matrix containing the appropriate powers of ω_n :

$$\begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \omega_n & \omega_n^2 & \omega_n^3 & \dots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \omega_n^6 & \dots & \omega_n^{2(n-1)} \\ 1 & \omega_n^3 & \omega_n^6 & \omega_n^9 & \dots & \omega_n^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \omega_n^{3(n-1)} & \dots & \omega_n^{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{pmatrix}.$$

The (k, j) entry of V_n is ω_n^{kj} , for $j, k = 0, 1, \dots, n-1$. The exponents of the entries of V_n form a multiplication table for factors 0 to $n-1$.

For the inverse operation, which we write as $a = \text{DFT}_n^{-1}(y)$, multiply y by the matrix V_n^{-1} , the inverse of V_n .

Theorem 30.7

For $j, k = 0, 1, \dots, n-1$, the (j, k) entry of V_n^{-1} is ω_n^{-jk}/n .

Proof We show that $V_n^{-1}V_n = I_n$, the $n \times n$ identity matrix. Consider the (k, k') entry of $V_n^{-1}V_n$:

$$\begin{aligned}
[V_n^{-1} V_n]_{kk'} &= \sum_{j=0}^{n-1} (\omega_n^{-jk} / n) (\omega_n^{jk'}) \\
&= \sum_{j=0}^{n-1} \omega_n^{j(k'-k)} / n .
\end{aligned}$$

This summation equals 1 if $k' = k$, and it is 0 otherwise by the summation lemma (Lemma 30.6). Note that in order for the summation lemma to apply, $k' - k$ must not be divisible by n . Indeed, it is not, since $-(n-1) \leq k' - k \leq n-1$.

■

With the inverse matrix V_n^{-1} defined, $\text{DFT}_n^{-1}(y)$ is given by

$$\begin{aligned}
a_j &= \sum_{k=0}^{n-1} y_k \frac{\omega_n^{-jk}}{n} \\
&= \frac{1}{n} \sum_{k=0}^{n-1} y_k \omega_n^{-kj}
\end{aligned} \tag{30.11}$$

for $j = 0, 1, \dots, n-1$. By comparing equations (30.8) and (30.11), you can see that if you modify the FFT algorithm to switch the roles of a and y , replace ω_n by ω_n^{-1} , and divide each element of the result by n , you get the inverse DFT (see Exercise 30.2-4). Thus, DFT_n^{-1} is computable in $\Theta(n \lg n)$ time as well.

Thus, the FFT and the inverse FFT provide a way to transform a polynomial of degree-bound n back and forth between its coefficient representation and a point-value representation in only $\Theta(n \lg n)$ time. In the context of polynomial multiplication, we have shown the following about the convolution $a \otimes b$ of vectors a and b :

Theorem 30.8 (Convolution theorem)

For any two vectors a and b of length n , where n is an exact power of 2,

$$a \otimes b = \text{DFT}_{2n}^{-1}(\text{DFT}_{2n}(a) \cdot \text{DFT}_{2n}(b)) ,$$

where the vectors a and b are padded with 0s to length $2n$ and $\cdot$ denotes the componentwise product of two $2n$ -element vectors.

■

Exercises

30.2-1

Prove Corollary 30.4.

30.2-2

Compute the DFT of the vector $(0, 1, 2, 3)$.

30.2-3

Do Exercise 30.1-1 by using the $\Theta(n \lg n)$ -time scheme.

30.2-4

Write pseudocode to compute DFT_n^{-1} in $\Theta(n \lg n)$ time.

30.2-5

Describe the generalization of the FFT procedure to the case in which n is an exact power of 3. Give a recurrence for the running time, and solve the recurrence.

★ 30.2-6

Instead of performing an n -element FFT over the field of complex numbers (where n is an exact power of 2), let's use the ring $\mathbb{Z}_m$ of integers modulo m , where $m = 2^{tn/2} + 1$ and t is an arbitrary positive integer. We can use $\omega = 2^t$ instead of ω_n as a principal n th root of unity, modulo m . Prove that the DFT and the inverse DFT are well defined in this system.

30.2-7

Given a list of values $z_0, z_1, \dots, z_{n-1}$ (possibly with repetitions), show how to find the coefficients of a polynomial $P(x)$ of degree-bound $n + 1$ that has zeros only at $z_0, z_1, \dots, z_{n-1}$ (possibly with repetitions). Your procedure should run in $O(n \lg^2 n)$ time. (*Hint:* The polynomial $P(x)$ has a zero at z_j if and only if $P(x)$ is a multiple of $(x - z_j)$.)

★ 30.2-8

The *chirp transform* of a vector $a = (a_0, a_1, \dots, a_{n-1})$ is the vector $y = (y_0, y_1, \dots, y_{n-1})$, where $y_k = \sum_{j=0}^{n-1} a_j z^{kj}$ and z is any complex number.

The DFT is therefore a special case of the chirp transform, obtained by taking $z = \omega_n$. Show how to evaluate the chirp transform for any complex number z in $O(n \lg n)$ time. (*Hint:* Use the equation

$$y_k = z^{k^2/2} \sum_{j=0}^{n-1} (a_j z^{j^2/2}) (z^{-(k-j)^2/2})$$

to view the chirp transform as a convolution.)

30.3 FFT circuits

Many of the FFT's applications in signal processing require the utmost speed, and so the FFT is often implemented as a circuit in hardware. The FFT's divide-and-conquer structure enables the circuit to have a parallel structure so that the *depth* of the circuit—the maximum number of computational elements between any output and any input that can reach it—is $\Theta(\lg n)$. Moreover, the structure of the FFT circuit has several interesting mathematical properties, which we won't go into here.

Butterfly operations

Notice that the **for** loop of lines 9–12 of the FFT procedure computes the value $\omega_n^k y_k^{\text{odd}}$ twice per iteration: once in line 10 and once in line 11. A good optimizing compiler produces code that evaluates this *common subexpression* just once, storing its value into a temporary variable, so that lines 10–11 are treated like the three lines

$$\begin{aligned} t &= \omega y_k^{\text{odd}} \\ y_k &= y_k^{\text{even}} + t \\ y_{k+(n/2)} &= y_k^{\text{even}} - t \end{aligned}$$

This operation, multiplying the twiddle factor $\omega = \omega_n^k$ by y_k^{odd} , storing the product into the temporary variable t , and adding and subtracting t from y_k^{even} , is known as a *butterfly operation*. Figure 30.3 shows it as a circuit, and you can see how it vaguely resembles the shape of a